

Technology Stack

Date	30 June 2026
Team ID	N/A
Project Name	Personalized Networking Assistant
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack:

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular, HTML/CSS)	Streamlit (Python)	Provides a simple and interactive user interface where users can enter profile details, event information, generate conversation starters, self-introductions, networking tips, perform fact verification, and view conversation history with minimal development effort.
2	Backend / Server-Side (e.g., Node.js, Python/Django)	FastAPI (Python)	Handles API requests, processes user inputs, integrates with the Groq API and Wikipedia API, manages SQLite operations, and provides high-performance REST APIs with automatic Swagger documentation.

	ngo, Java)		
3	Database / Data Storage (e.g., MySQL, MongoDB, PostgreSQL)	SQLite (networking.db)	Stores conversation history locally in a lightweight relational database. SQLite requires no separate database server and is suitable for local development and demonstration.
4	Cloud / Hosting / Deployment (e.g., AWS, Azure, Heroku, Docker)	Local Deployment using Uvicorn & Streamlit	Runs FastAPI backend using Uvicorn and Streamlit frontend locally for development, testing, and demonstration without cloud infrastructure.
5	Version Control & CI/CD (e.g., GitHub, GitLab, Jenkins)	Git & GitHub	Tracks source code changes, enables collaboration among team members, maintains version history, and supports project management.
6	Third-Party APIs / Other Tools (e.g., Stripe, Twilio, Google Maps)	Groq API, Wikipedia API, PyTest, HTTPX TestClient, Pydantic, python-dotenv	Groq API generates AI-powered networking responses, Wikipedia API provides fact verification, PyTest and HTTPX TestClient are used for unit and integration testing, Pydantic validates request/response data, and python-dotenv securely manages API keys through environment variables.